Sparse Matrix Multiplication Accelerator

Praneeth Kodavati

cs24btech11034

November 27, 2025

Abstract

This project introduces a new instruction, `sparse_mul`, into the existing ISA of the RISC-V simulator. This instruction accepts four operands: `rs1`, `rs2`, `rs3`, and `rd`. The left-hand matrix in this multiplication can be sparse, meaning that a large number of its entries are zeros. To compute the resultant matrix efficiently, the instruction uses the metadata of the sparse matrix to skip computations involving zeros, significantly reducing computation time for highly sparse matrices. All this computation is performed in a dedicated accelerator.

Instruction Encoding

As there is no Instruction Format which has 4 operands, this project added a new instruction format SM type standing for Sparse Multiplication in which the instruction `sparse_mul` is included. The Instruction Encoding is as follows.

[31–30]	**[29–25]**	**[24–20]**	**[19–15]**	**[14–12]**	**[11–7]**	**[6–0]**
`func2`	`reg3`	`reg2`	`reg1`	`func3`	`rd`	`opcode`
00	—	—	—	000	—	1011011

Table 1: SM-Type instruction format for `sparse_mul`.

In the `sparse_mul` instruction, the registers are used as follows:

- `rs1` (**reg1**) holds the base address of the **sparse matrix**.
- `rs2` (**reg2**) holds the base address of the **dense matrix**.
- `rs3` (**reg3**) holds the **metadata** corresponding to the sparse matrix.
- `rd` is the **destination register** that stores the base address of the resulting matrix.

The registers can be any integer from 0 to 31.

Processor Design Overview

The given simulator implements a single-cycle processor, meaning that each instruction from the ISA is executed within a single clock period. However, executing the `sparse_mul` instruction within one cycle is not feasible due to its computational complexity. To address this, the project introduces a separate datapath dedicated to the execution of the `sparse_mul` instruction. This datapath functions as a four-stage pipelined processor.

Two clocks are defined for the system, each with different clock periods. The first clock corresponds to the main processor and has a period equal to the longest instruction delay among the existing ISA instructions (excluding the SM-type). The second clock corresponds to the pipelined datapath and has a period determined by the slowest of the four stages.

Since the main datapath operates as a single-cycle design, the instruction fetch unit does not fetch the next instruction until the current one has completed execution. As matrix multiplication is computationally intensive, especially for large matrices, this results in significant waiting time for subsequent instructions in the instruction memory. Further discussion about this is mentioned later in the report.

Matrix Representation in Data Memory

Each entry in the matrix is assumed to be a 64-bit integer value. Hence, to store a matrix of size $n \times m$, a total of $n \times m \times 8$ bytes of data memory is required.

In the data memory, we store the matrix in a linear (one-dimensional) form. This means that all the elements of the matrix are stored one after another in a single continuous block of memory. The reason for this is that the memory in a computer is linear by nature, it does not store data in rows and columns like how we write matrices on paper.

By storing the matrix linearly, it becomes easier to calculate the memory address of any element using its row and column index. This also helps in faster access during sparse computations.

Problem Faced

Matrix multiplication is not the same as a simple instruction like adding two integers. In this case, both matrices must be compatible for multiplication. That means the number of rows and columns should not be non-positive and the number of columns in the left matrix should be equal to the number of rows in the right matrix. If the processor does not check this condition, it can lead to a software crash. To avoid this, the project adds a **compatibility detection unit** which first checks the compatibility of the matrices and then sends them to the sparse pipeline.

The compatibility checker must be able to read the dimensions of both matrices. Initially, an attempt was made to store the dimensions inside the ISA encoding of `sparse_mul`, but this was not possible due to a lack of space. Since the architecture follows the RISC design, each instruction has a fixed length of 32 bits, leaving no room for additional information. To handle this limitation, the representation of matrices was modified as explained below.

Final Matrix Representation

To resolve this issue, the matrix representation is slightly modified. The first 4 bytes of the matrix store the number of rows, and the next 4 bytes store the number of columns. This allows a matrix to be as large as $2^{32} - 1 \times 2^{32} - 1$ (This was reduced even more due to some design decisions which were made later), which is sufficiently large for practical use. Both values are treated as unsigned since negative dimensions do not make sense in this context. With this representation, the compatibility checker can easily read the dimensions of both matrices and decide whether to forward them to the sparse pipeline or raise an error.

Choosing correct algorithm for sparse matrix multiplication

Matrix Multiplication Algorithms

Matrix multiplication can be implemented in several structural ways depending on which portions of the matrices are read together. Some common traversal-based approaches are listed below.

1. **Row–Column Multiplication:** For each row i of matrix A and each column j of matrix B, compute the dot product of the entire row of A with the entire column of B.

This Algorithm is not efficient because we should load all values from sparse matrix where most of the entries are zero.

2. **Row–Row (Chosen Algorithm):** We can skip most of the zero entries by doing in this algorithm.

   For each row i of matrix A, iterate through its non-zero entries. For each non-zero element $A[i][j]$, fetch the jth row of matrix B, multiply that row by the scalar value $A[i][j]$, and accumulate the result into a temporary vector representing the ith row of the output matrix C. After finishing all non-zero elements in row i, write this accumulated vector to row i of C. The temporary vector serving as accumulator is initially set to zero.

3. **Column–Column Multiplication:** This method computes each column of C by iterating over the corresponding columns of A and B. It is rarely used in hardware as it requires strided access to memory and frequent jumps between non-contiguous memory locations.

The **row–row method** (Algorithm 2) is particularly efficient when matrix A is sparse because it completely skips over zero entries. Memory fetches are done only for non-zero elements, and each non-zero element directly corresponds to one row access in matrix B. Since memory access is the slowest operation in most architectures, avoiding zero fetches yields significant performance benefits. Additionally, the approach maintains sequential row-wise access to both matrices, making it suitable for hardware pipelines.

—

Illustration

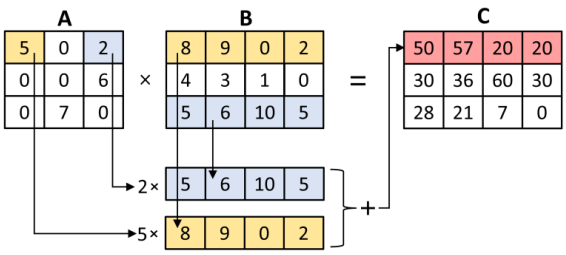

Figure 1: An example of row-row style matrix multiplication

Intuition Behind Metadata

In sparse matrix multiplication, most entries in the left matrix are zeros. If the processor fetches every element sequentially from memory, including zeros, a huge amount of time is wasted on unnecessary reads. Hence, the main goal of the metadata is to help the processor **skip over the zero elements** efficiently, while still being able to determine the exact position of the next non-zero entry.

A very basic version of metadata could be an array of bits, where each bit represents whether a corresponding element in the matrix is zero or non-zero (for example, `1` for non-zero and `0` for zero). Although simple, this approach becomes extremely inefficient for large matrices because

it still requires one bit per element, resulting in significant memory usage and bandwidth waste.

To reduce this overhead, a better design is to store **offsets** instead of absolute positions. That means, instead of storing where each non-zero value is, we only store how far the next non-zero element is from the current one. This allows the processor to "jump" directly to the next useful entry, skipping all intermediate zeros. If metadata store absolutes column indices of non zero elements, there is a good scope for ambiguity which users don't want. For example if the next non zero value is at column index 7 according to metadata and the current non zero value is at index 3 there is no information available on row index of next non zero element. It can be in the same row or the next row or the next to next row. This problem is countered by using offset metadata.

However, since the matrix is two-dimensional, the metadata must also indicate when a new row starts. This is where the most significant bit (MSB) can be used as a flag: when the MSB is set, it indicates that the next non-zero element is in the next row. To put it simply, if the current offset has 1 in MSB then that implies the current non zero element is the last element in it's row. Otherwise, the remaining bits specify the offset within the current row. This approach keeps the representation compact and helps the hardware traverse sparse matrices efficiently without needing any external row or column tracking structure.

Now the challenge is what Offset should be given at the last element of a row? should it be the number of elements required to skip for the next non zero value? If such a metadata is taken it will create some issues. For example, if an entire row is filled with zero, the sparse pipeline will directly read non zero values from next row and will skip this row. So for rows which have complete zeroes, no output is being written to memory.

If the memory consists some garbage there instead of zeroes , it will retain those values, which is a mistake as zero row should have been written to memory. To overcome these the offset will always be provided such that the next element is the first element of next row even if it's non zero.

The MSB helps in several other things, as this concludes the row, we can write the accumulated row to the data memory now and set it back to zero and even set the current column index to zero, increment row index. As the maximum possible offset is now $2^{31} - 1$ and matrix can be as large as $2^{31} - 1 \times 2^{31} - 1$.

Sparse Matrix Multiplication Pipeline

In a 5-stage pipelined processor instructions run faster due to sharing of hardware there by increasing throughput.
One iteration which start at stage-1 and ends at stage-5 is called an instruction. In this pipeline we shouldn't call it as instruction and hence let's call it as **work unit**.

Work unit

In a work unit, the function fetches a non zero value in sparse-matrix. Let the column index of these values is 'i'. Now fetch the row i of dense matrix. Multiply the scalar number with row i of dense matrix. Add this row to an accumulator, if the current element is last non-zero in its row then this accumulator is written to the output buffer. In this implementation, these work units have been pipelined to reduce cycles and increase through-put.

Storage unit

The **Storage Unit** is a hardware block that holds all the runtime states needed by the sparse-dense multiplication pipeline. It will contain base addresses of matrices and they get updated every cycle.

The Storage Unit keeps track of:

- **Base addresses** for the sparse metadata region, sparse value region, dense matrix rows, and the destination block. These act as starting pointers for the different memory streams.
- **Temporary address cursors** that move through the sparse structure as the pipeline advances. These pointers behave like internal address generators and ensure that every pipeline stage knows where to fetch its next data word.
- **Dimension parameters** such as the number of sparse rows, sparse columns, dense rows, and dense columns. These define the iteration boundaries for the multiplication block.
- **Current processing state**, including the active row, active column, active lane, and a completion flag. These act like loop counters in hardware and guide the per-cycle flow.
- **Accumulator vector**, which stores the partial sum of the ongoing dense output row. Each lane in this vector corresponds to one element of the output being accumulated across cycles.
- **Write-back buffer**, which holds completed output vectors. This ensures that memory writes do not stall the pipeline and can occur whenever the memory controller is ready.

The short-forms for the 4 pipeline stages are OF, VF, SV, ACC.

Why Storage unit is needed ? For stages like VF, SV, ACC to do their computation they will get thier input values and control signals via Pipelined registers in between them. The 3 piplined registers are OF/VF, VF/SV , SV/ACC. But what about the first stage OF ? OF stage is getting it's base values from Storage unit which it updates cycle to cycle.

Pipeline Registers

The pipeline uses three inter-stage registers to transfer state and data between consecutive stages. Each register captures the output of one stage at the rising clock edge and presents it as input to the next stage in the following cycle.

1. OF/VF Register

Purpose: To pass updated values of a work unit to VF stage.

This register consists the following:

- a 1-bit signal isValid
- a 1-bit signal isLast
- a 32-bit number indicating row index of the current work unit.
- a 32-bit number indicating column index of the current work unit.
- a 32-bit number indicating lane number of the current work unit.
- a 64-bit address indicating the address of the non-zero sparse value of the current work unit.

2. VF/SV Register

Purpose: To pass updated values of a work unit to SV stage.
This register consists the following:

- a 1-bit signal isValid
- a 1-bit signal isLast
- a 32-bit number indicating row index of the current work unit.
- a 32-bit number indicating column index of the current work unit.
- a 32-bit number indicating lane number of the current work unit.
- entire row of dense matrix whose row-index is column index of sparse value.
- a 64-bit number - the non zero sparse matrix value of this work unit.

3. SV–ACC Register

Purpose: To pass updated values of a work unit to ACC stage.
This register consists the following:

- a 1-bit signal isValid
- a 1-bit signal isLast
- a 32-bit number indicating row index of the current work unit.
- a 32-bit number indicating column index of the current work unit.
- a 32-bit number indicating lane number of the current work unit.
- resultant row/vector obtained after doing scalar multiplication of dense row with sparse matrix value in svmul stage.

Summary of Register Flow

- **Stage-to-Stage Path:** OF → VF → SV → ACC

Pipeline Stages

This section describes the four-stage pipeline used in the sparse–dense multiplication engine. Each stage performs a specific hardware function and passes its results to the next stage through well-defined pipeline registers.

What's this lane ?

The number of columns in dense matrix can be very large , hence loading an entire row is not ideal to do(In a pipeline time taken for each of the stages should be as close as possible, so we can't afford uncertainty). A dense matrix of dimensions m x n can be broken into (n+7)/8 matrices each with dimensions as m x 8. Now as the number of columns are just 8 loading doesn't need a large interrupt time. The final result is just concatenation of these (n+7)/8 result matrices. So here lane represents number of smaller resultant matrices done before this work unit.

—

Illustration

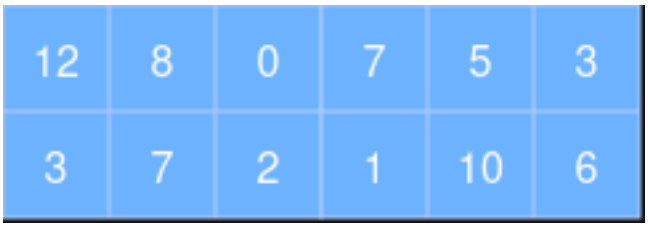

Figure 2: Normal matrix

Figure 3: matrix can be split along a column and output can still be achieved by multiplying with the 2 matrices independently and concatenating the outputs side by side to get the actual output.

1. Offset Fetch (OF) Stage

This stage is designed for updating OF/VF register correctly. This stage checks whether the current work unit is last, if it is last then it will turn isLast of OF/VF to true. after updating the OF/VF register values, this will update base values of storage unit with the offset it read in this work unit. This is to get ready for next work unit getting in.

2. Vector Fetch (VF) Stage

In this stage, the accelerator reads a vector of length 8 from a dense matrix and simultaneously writes a vector(if present) from writebackbuffer to the output matrix. After this the accelerator also reads a scalar from the sparse matrix.

3. Scalar–Vector Multiply (SV) Stage

In this stage the accelerator multiplies the dense row vector with the sparse matrix scalar. The accelerator uses SIMD parallel multipliers to perform this multiplication.

4. Accumulate (ACC) Stage

This stage accumulates all resultant vectors. If its the last work unit of the row then accelerator stores this accumulated vector to writebackbuffer and sets accumulator back to zero.

Datapath Overview

The Normal instructions are run on regular datapath but not on accelerator. When a sparse_mul instruction is encountered (detected in decode phase) the instruction is then sent to accelerator and waits for the accelerator to complete it and main datapath fetches no other instruction. The amount of time/cycles for which the main datapath remained idle will depend on the size of the matrices hence unpredictable before execution. The time taken by the second stage of this pipeline is approximately 1.5X of the time taken by load instruction in main datapath where both are slowest units in their respective data paths. Hence we decide the clock period of second clock as twice of first clock period. So if it takes x clock cycles in accelerator then main datapath is stalled for 2x cycles.

Meta instruction

Computing Metadata matrix of a sparse matrix everytime manually is painful at times. Hence this simulator supports one more multicycle instruction metadata (use case: metadata rd, rs1, x0). This instruction takes the matrix in rs1 and stores it's metadata in the address stored in rd. This instruction doesn't focus much on latency as we use this instruction only once per a sparse matrix and can reuse the meta data matrix again and again. This instruction runs using the main clock itself. The number of clock cycles it needs to create metadata depends on dimensions on given matrix.

Peroformance Analysis

A normal code in RISC-V takes 8(abc) cycles approximately to multiply 2 matrices of dimensions axb and bxc where as this accelerator takes $\frac{abc(1-s)}{4}$ clock-1 cycles where ab represents number of elements in sparse matrix and s is sparsity index. ab(1-s) represent number of non-zero elements and as we perform this for c/8 times the number of work units are abc(1-s)/8. For large matrices it takes approximately $\frac{abc(1-s)}{8}$ cycles to run those work units. This is equivalent to $\frac{abc(1-s)}{4}$ clock-1 cycles.

Note: The accelerator also treats first column values as work units always but we didnt consider it in performance analysis. So there is a slight uncertainty about number of work units. In worst case number of work units will be $\frac{(a+ab(1-s))c}{4}$. The amount of speed up we get depends on how large 'a' is in the worst case scenario.

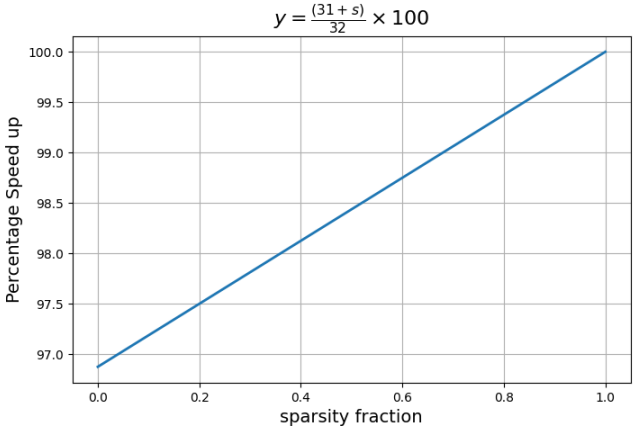

Figure 4: Plot for percentage speedup vs sparsity index(subjected to change for different matrices)

Testing Framework

Verifying the correctness of matrix multiplication in a hardware model is challenging due to the scale of the computation and the sparsity patterns involved. Manual inspection of results is infeasible, especially when the matrices become large. To address this, we designed an automated testing framework that systematically generates inputs, simulates the hardware and validates the output.

Randomized Test Generation. A C++ program (`tester.cpp`) is used to generate randomized test cases. For each test:

- Random dimensions (a, b, c) are selected under a global constraint on the total operation count $\sum abc \leq$ `MAX_OPS`.
- Two matrices $A \in \mathbb{Z}^{a\times b}$ and $B \in \mathbb{Z}^{b\times c}$ are constructed with configurable sparsity.
- The expected product $C = A \times B$ is computed using cpp function.

Automatic Assembly Patching. The program then flattens the matrices into row-major order and embeds them directly into an assembly file (`test.s`) by replacing the labels `array1:` and `array2:` with newly constructed `.dword` lists containing:

- a packed header encoding the matrix dimensions
- the flattened elements of each matrix.

Hardware Execution. Once patched, the test file is executed on the RISC-V virtual machine:

```
./vm --run test.s
```

The hardware output is redirected into `tester_out.txt`.

Output Verification. The program reads back the hardware-generated output and compares it with the software-computed expected result. If the vectors match exactly, the test is marked as passed; otherwise, the program reports:

- the matrix dimensions,
- the expected number of outputs versus what was received,
- and a side-by-side printout of the expected and observed results.

This automated pipeline allows large numbers of randomized test cases to be validated efficiently. It ensures correctness across a wide range of sparsity patterns, dimensions, and edge cases, providing a high level of confidence in the hardware matrix multiplication implementation. To use this tester go through the readme.md.

Hardware Cost of Accelerator

Conventions Used

- All datapaths are 64-bit wide (HFT-grade arithmetic).
- Technology node assumed: 7nm FinFET (TSMC-class).
- Area numbers used:
  - 64-bit signed multiplier: $\approx 12{,}000\,\mu m^2$
  - 64-bit signed adder: $\approx 400\,\mu m^2$
  - 64-bit register (1 flop): $\approx 4\,\mu m^2$
  - 64-bit SRAM (1 KB macro): $\approx 6{,}500\,\mu m^2$
  - Control FSM (per small stage): $\approx 300\,\mu m^2$

Stage: Offset Fetch

What we do here:

- Read compressed sparse offset (32-bit).
- Extract MSB for "isLast".
- Extract lower bits for column advancement.
- Update pointers: sparse address, metadata address, row/column counters.

Required hardware components:

- 32-bit register for metadata word
- Comparator for MSB extraction
- Adders to update: row, column, meta pointer, sparse pointer
- Small FSM for control

Area:

$$A_{\text{offset}} = (4 \text{ adders} \times 400) + (1 \times 300) \approx 1{,}900\,\mu m^2$$

Stage: Vector Fetch

What we do here:

- Read 64-bit scalar value from sparse matrix.
- Read 8-lane dense vector (each 64-bit).
- Handle write-back buffer draining.

Required hardware components:

- 64-bit register for scalar
- 8×64-bit vector register
- Memory read interface (SRAM bus)
- Small FSM for buffer handling

Area:

$$A_{\text{vf}} = (9 \times 64\text{-bit registers} \times 4) + 300 \approx 2{,}600\,\mu m^2$$

Stage: SVMUL (SIMD Scalar–Vector Multiply)

What we do here:

- Multiply scalar (64-bit) with each element of the vector (8 lanes).
- This uses 8 fully parallel 64-bit multipliers (HFT-grade SIMD).

Required hardware components:

- 8×64-bit signed multipliers
- Pipeline registers for throughput
- Lane mixer and result bus

$$A_{\text{svmul}} = (8 \times 12{,}000) + (8 \times 64 \times 4) \approx 97{,}000\,\mu m^2$$

Stage: ACC (Vector Accumulation)

What we do here:

- Add partial SIMD products into accumulator.
- Each lane needs a 64-bit adder.

Required hardware components:

- 8×64-bit adders
- 8-lane accumulator register

$$A_{\text{acc}} = (8 \times 400) + (8 \times 64 \times 4) \approx 6{,}000\,\mu m^2$$

Stage: Write-Back

What we do here:

- Mask lanes based on writeControls().
- Write dense row results to memory.

Required hardware components:

- 8×1-bit mask flops
- 8-lane write datapath (64-bit)
- Address generator adder

$$A_{\text{wb}} = (8 \times 1) + (8 \times 64 \times 4) + 400 \approx 2{,}200\,\mu m^2$$

Clocking + Stall Control (Separate Accelerator Clock)

What is required:

- Independent PLL/clock-domain generator for accelerator.
- Clock gating for 5 pipeline stages.
- Handshake logic for stalling the main CPU pipeline (valid/ready protocol + boundary synchronizers).

Area:

$$A_{\text{clock}} = \text{PLL core } (12{,}000\,\mu m^2) + \text{clock-tree buffers } (5{,}000\,\mu m^2) + \text{synchronizers } (1{,}000\,\mu m^2) \approx 18{,}000\,\mu m^2$$

Total Accelerator Die Area

$$A_{\text{total}} = A_{\text{offset}} + A_{\text{vf}} + A_{\text{svmul}} + A_{\text{acc}} + A_{\text{wb}} + A_{\text{clock}}$$

$$A_{\text{total}} = 1{,}900 + 2{,}600 + 97{,}000 + 6{,}000 + 2{,}200 + 18{,}000$$

$$\boxed{A_{\text{total}} \approx 127{,}700\,\mu m^2 \approx 0.13\ \text{mm}^2}$$